8 CONCLUSION

In this chapter we briefly review the results achieved in this work, and propose some directions for future work and investigations.

8.1 RESULTS ACHIEVED

8.1.1 Updating QA-Prolog

The QA-Prolog software stack had remained without maintenance while the frameworks on which it relied were updated. As a result, the pipeline initially did not work and did not correctly interface with the D-Wave Ocean framework.

Part of the work of this thesis consisted in restoring compatibility between the software written *ad hoc* for the QA-Prolog pipeline and the other software used to complete the entire translation process.

At present, the updated version of QMASM is available[1] and allows the use of the entire QA-Prolog pipeline, that is, starting from a Prolog program, obtaining an equivalent problem in QUBO form, and retrieving the results in a readable format after solving the QUBO problem.

8.1.2 Ontology in Prolog

The work of this thesis continued with the translation of an ontology expressed in OWL (Section 3.1.3) into an equivalent knowledge base expressed in Prolog. We started by following the literature and implementing the main concepts offered by OWL in Prolog. We focused on the concepts of class and subclass, relations between individuals, and the identification of inconsistencies in the KB. We also mentioned that in the literature there are works showing how to translate more advanced concepts such as the open world assumption and enumerable classes.

QA-Prolog does not implement all the functionalities of Prolog; the main limitation is that it does not allow the definition of recursive rules. The contribution of this thesis was, therefore, to propose solutions to overcome the restrictions imposed by QA-Prolog, managing to obtain a KB equivalent to the original ontology and compatible with the QA-Prolog pipeline.

The proposed solution to address the problem of recursive calls was to perform recursion unfolding. We then focused on the drawbacks of this technique. We observed that developing many levels of recursion through unfolding leads to a significant increase in the size of the problem, and therefore in the number of *qubits* required to represent it. We estimated and

1 a https://github.com/DavideCamino/qmasm.

discussed the quadratic increase in the size of the problem as a function of the number of unfolded recursion levels and observed that this can be problematic considering the current state of quantum machines, both gate-based and quantum annealers.

8.1.3 QAOA Algorithm

The last chapter of the thesis presents an alternative algorithm to annealing for solving an optimization problem in QUBO form such as the one obtained at the end of the QA-Prolog pipeline.

We presented the QAOA algorithm, briefly discussing its characteristics from a theoretical point of view. We then focused on automating the rewriting from the QUBO weight matrix (Section 2.3.3) to a format compatible with IBM's Qiskit framework.

After obtaining the translation, we performed some tests starting from the output of QA-Prolog and attempting to find the solution through QAOA. We carried out various tests and commented on the obtained results, observing that QAOA is not always able to return the correct solution.

8.1.4 A rewriting process

This work can be summarized as a long chain of rewritings of problems into equivalent problems. The goal is to move from one format to another, from one language to another, while preserving certain properties of interest, the most important one, of course, is that the solution of the problem must never change.

This rewriting process makes it possible to have different versions of the same problem, to observe it from different perspectives, and to choose the most convenient format to manipulate the problem depending on the objective. For example, we would like to describe the problem in the most convenient way possible, using a high-level language; on the other hand, we would like to solve it as quickly as possible, and if this means using a quantum machine, we would like to be able to move from one representation to another easily. All intermediate steps are also important because they offer different views of the problem and give us the possibility to extract properties that are not obvious at other levels of abstraction and, consequently, to optimize the problem or identify the best solving strategy.

8.2 FUTURE WORK

We present some directions for future research that can benefit from the tools developed in this work and from the results achieved.

8.2.1 Testing on quantum hardware

In this thesis we limited ourselves to testing the proposed algorithms and solutions on the simulators provided by D-Wave and IBM. The reasons for

this choice are mainly related to licensing issues. D-Wave does not allow the use of its platforms without a subscription; IBM instead offers a certain amount of computational time on its quantum machines, but without a paid plan the priority assigned to tasks is minimal, and due to time constraints it was not possible to perform satisfactory tests on quantum computers.

A possible future direction would be to verify the experimental results achieved in this thesis also on quantum hardware. On one hand, we can expect a speedup in performance due to the exploitation of quantum properties rather than their simulation on classical hardware. On the other hand, an improvement in the precision of the obtained solutions is not sure, due to noise issues that are still significant in current machines.

8.2.2 Increasing QMASM compatibility

QMASM (Section 5.2.1) is the last stage of the QA-Prolog pipeline and was developed to interface with the D-Wave Ocean framework. Thanks to QMASM it is therefore possible to submit a problem in QUBO form directly to D-Wave quantum annealers or to solve it using one of the simulators provided within the Ocean framework. These simulators are designed to test code on small instances in order to be more confident in correct execution when moving to quantum hardware. In our experiments we encountered the limitations of these simulators in Section 5.2.3.

Making QMASM compatible with other frameworks and libraries for solving QUBO problems would allow for greater flexibility in choosing the solving tool, potentially enabling both classical and quantum approaches. This would open the possibility of using simulated annealing, simulated quantum annealing, or the QAOA algorithm, in addition to quantum annealing. It would also remove the dependency on the D-Wave framework.

Interfacing QMASM with other tools is an approach fundamentally different from the one adopted in Chapter 7. There, we started from the matrix representation of the QUBO problem and translated it into a format compatible with Qiskit, thus ending all interactions with QMASM. If QMASM itself were able to interface with other solvers, we could also benefit from traversing the pipeline in reverse, that is, once the results are obtained, translating them back into a high-level language to facilitate interpretation.

In Chapter 7 we chose to implement the QAOA algorithm in Qiskit to solve the satisfiability problem, since we could directly read the solution from the final configuration of the *qubits*. Encoding a KB and a query would have been possible, but interpreting the results in binary format would have been extremely complex. If QMASM interfaced with Qiskit in the same way it does with Ocean, it would be possible to obtain results in a human-readable format.